Evaluación Final – Desafío del Curso

DEL ARCHIVADOR AL ALGORITMO, UN PROTOTIPO PARA LA MESA DE PARTES DIGITAL

Integrantes:

- Graciela Lizet Ruiz Ramos.
- Aaron Keneth Gonzales Cortez.
- Alejandro Miguel Huilcaya Dominguez.
- Joan Narciso Villar.
- Manuel Anthony Ruiz Perez.

Universidad Privada del Norte

CIIN1205P: FUNDAMENTOS DE PROGRAMACIÓN

Prof. Percy Angel Pardo Zapata

20 de octubre del 2026

Contexto de la situación:

Lima Metropolitana es una megalópolis caracterizada por su vasta extensión territorial y una alta complejidad urbana. Diariamente, millones de ciudadanos se enfrentan a uno de los mayores índices de congestión vehicular de la región. En este escenario, el modelo tradicional de atención presencial en la Municipalidad de Lima representa un desafío logístico significativo; para un residente que se traslada desde distritos periféricos hasta el Cercado de Lima, presentar un simple documento o solicitud puede implicar la pérdida de medio día de trabajo, gastos de transporte y el desgaste físico de lidiar con el tráfico y las largas filas.

Además de las barreras geográficas, el sistema de atención física está sujeto a horarios de oficina rígidos (generalmente de lunes a viernes en horario diurno). Esto genera cuellos de botella administrativos, retrasos en la atención y una percepción ciudadana de burocracia lenta y desconectada de las necesidades de la población moderna.

Frente a esta problemática y en el marco de la transformación digital del Estado, surge la necesidad imperativa de desarrollar la **Mesa de Partes Virtual de la Municipalidad de Lima**. Este proyecto nace con el propósito de romper las barreras físicas y temporales entre el gobierno local y el ciudadano, democratizando el acceso a los servicios municipales.

La creación de esta plataforma se fundamenta en los siguientes pilares estratégicos:

- **Accesibilidad y Disponibilidad 24/7:** La plataforma permitirá a los ciudadanos ingresar documentos, solicitudes y expedientes en cualquier momento del día, desde cualquier dispositivo con conexión a internet, eliminando la dependencia de los horarios de oficina y evitando desplazamientos innecesarios.
- **Transparencia y Trazabilidad:** Al digitalizar el proceso de entrada, el sistema generará un número de expediente automático, permitiendo al ciudadano hacer un seguimiento en tiempo real del estado de su trámite. Esto reduce la incertidumbre, minimiza el riesgo de pérdida de documentos y fomenta la confianza en la gestión pública.
- **Agilidad Administrativa:** La automatización en la recepción y derivación de documentos optimizará los flujos de trabajo internos (procesos de negocio) de la municipalidad, reduciendo los tiempos de respuesta y aliviando la carga operativa del personal de atención al cliente.
- **Sostenibilidad y Ecoeficiencia:** El proyecto se alinea con las políticas de "Cero Papel" en la administración pública, reduciendo drásticamente el consumo de recursos impresos y disminuyendo la huella de carbono asociada al traslado físico de los ciudadanos.

1. Observación del proceso real.

1.1 Justificación de la elección.

Se seleccionó el área de trámite documentario (Mesa de Partes presencial) de la Municipalidad Metropolitana de Lima para esta observación debido a la alta afluencia diaria de ciudadanos y la complejidad logística que implica para los residentes trasladarse desde distintos puntos de la ciudad. Observar este proceso físico es fundamental para identificar los cuellos de botella actuales (largos tiempos de espera, uso intensivo de papel y limitación de horarios) que justifican la necesidad de implementar una alternativa virtual.

1.2 Bitácora fotográfica y notas de campo

Las evidencias visuales fechadas que respaldan esta observación han sido recopiladas en un drive por separado. (Anexo 1)

1.3 Descripción del proceso observado

Durante la visita presencial, se observó que el registro de documentos que ingresan sigue este flujo (Anexo 2):

- **Paso 1 - Llegada y espera:**

  El ciudadano ingresa a la sede y realiza una fila de atención. Tiempo de espera aproximado: *5 -10 min*

- **Paso 2 – Llenado y fotocopia del formulario:**

  Le hacen entrega de un formulario respectivo al cual debe de llenar, se dio por hecho ese momento, pero le hicieron la recomendación de que sacara fotocopia, el ciudadano sale para buscar un lugar donde hagan ese acto y poder regresar.

- **Paso 3 – Retorno y obtención del Ticket:**

  El ciudadano retorna a la municipalidad con la copia y el formulario respectivo y se continuo el proceso donde obtuvo un ticket para ser atendido. Tiempo de espera aproximado: *15 a 20 min*

- **Paso 4 – Atención en ventanilla:**

  El personal revisa el documentario, si esta mal hace que hagas un nuevo formulario (rehacer los pasos anteriores), caso contrario re rosigue con el siguiente paso.

- **Paso 5 – Registro de la solicitud:**

  El trabajador realizo preguntas al ciudadano para completar con la información proporcionada, después de ello se registro la solicitud en la computadora.
- **Paso 6 – Tiempo de respuesta:**
  Después de haber registrado la solicitud se le dio de información al ciudadano que tendría una respuesta aproximadamente de hasta 30 días.
- **Paso 7 – Finalización de la visita:**
  La visita termino a las 11:08 am, por lo tanto, todo el transcurso se dio una demora de 1 hora y 3 minutos desde el momento que se inició todo.

2. Levantamientos de requerimientos

En esta parte no se pudo dar una entrevista con el área en común por un tema de políticas y horario laboral.

Pero, durante la visita de campo, se logró identificar diversas deficiencias operativas y cuellos de botella en el proceso presencial. Estas situaciones evidencian las limitaciones del sistema actual y justifican el desarrollo de la plataforma virtual. Los hallazgos se han clasificado de la siguiente manera:

A. Deficiencias en la Eficiencia Operativa

- **Dependencia de formatos manuales:** El requerimiento de llenar formularios manuscritos ralentiza el proceso de recepción y recepción de expedientes.
- **Descoordinación en el flujo de atención:** Existen problemas de comunicación e información entre el filtro inicial (recepción) y la ventanilla final (mesa de partes), lo que genera confusión en los pasos que debe seguir el usuario.
- **Reprocesos administrativos:** Se detectó la necesidad de reiniciar o detener trámites en curso debido a la validación tardía de información física incongruente o incompleta.

B. Impacto Negativo en la Experiencia del Ciudadano

- **Tiempos de espera prolongados:** Se registran demoras continuas en ventanilla de 20 minutos a más, tiempo que fluctúa de manera impredecible según la afluencia de personas.
- **Carga logística adicional:** El usuario se ve en la obligación de abandonar las instalaciones temporalmente para tramitar copias físicas (fotocopias) exigidas como requisitos, interrumpiendo su turno y atención.

C. Vulnerabilidades en la Gestión de la Información

- **Alta incidencia de errores de digitación (factor humano):** La transcripción manual de los datos desde el papel hacia el sistema de la institución genera equivocaciones recurrentes. Durante la observación, se evidenció que este proceso manual resulta en nombres mal registrados en los documentos oficiales de respuesta.
- **Carencia de trazabilidad:** El ciudadano se enfrenta a una total "ceguera" sobre el estado de su solicitud una vez entregada. Al no existir una plataforma digital integrada, es imposible realizar un seguimiento autónomo (*tracking*) del avance del trámite.

3. Análisis de alternativas de Solución.

3.1 Cuadro comparativo de soluciones algorítmicas posibles.

Criterios de Evaluación	Solución 1: Desarrollo Propio en Python (Propuesta del Equipo)	Solución 2: Automatización con Plataformas de Low-Code/Formularios.	Solución 3: Alternativa de Tercerización (Contratación Externa)
Requerimientos	Levantamiento interno de datos, identificación exacta de entradas, procesos y salidas, y definición de los requisitos del sistema diseñados a medida.	Adaptación de los flujos y procesos de la municipalidad a las limitaciones y plantillas de una plataforma comercial preexistente.	Elaboración de un pliego de requerimientos estricto (Términos de Referencia) y contratos de confidencialidad para el proveedor externo.
Recursos	Tiempo del equipo distribuido mediante la asignación de responsabilidades, capacidad de análisis de problemas y conocimientos de programación en Python.	Pago de licencias de software por usuario (suscripciones mensuales) y tiempo de configuración básica.	Alto presupuesto económico para pagar los honorarios de la empresa desarrolladora (Software Factory) y tiempo para supervisión legal.
Viabilidad	**Alta.** Permite un control total sobre la evaluación de las posibilidades algorítmicas, el diseño del flujo del proyecto y no requiere inversión de capital monetario, solo esfuerzo intelectual.	**Media.** Es rápida de implementar a corto plazo, pero presenta baja viabilidad técnica para escalar o personalizar módulos específicos a largo plazo.	**Baja.** Inviable para el contexto del equipo debido a los altos costos financieros involucrados y la delegación total del aprendizaje técnico a un tercero.

Fuente: Elaboración propia con base a los análisis de alternativas de solución.

Solución ganadora: Solución 1

Esta decisión se fundamenta en la viabilidad técnica y los recursos actuales del equipo de trabajo. Al optar por una construcción algoritmia interna, se facilita la ejecución directa de las actividades planificadas tras la visita de campo, tales como el ordenamiento de los datos obtenidos y la identificación estructurada de las entradas, procesos y salidas.

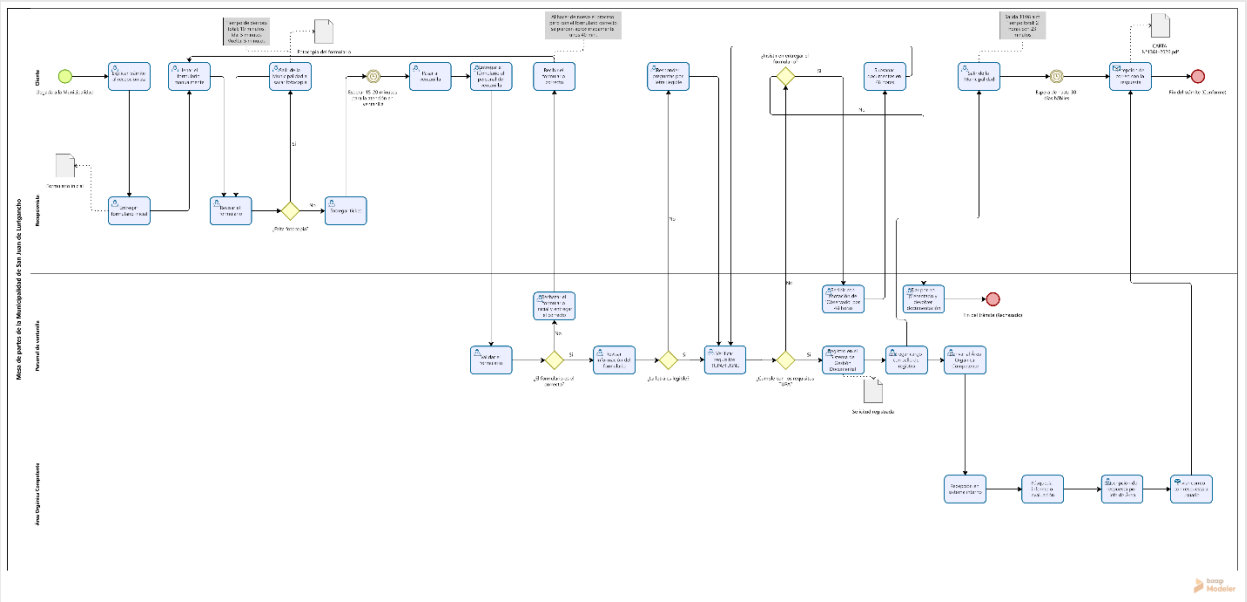
A diferencia de la tercerización, que transfiere el desarrollo a un agente externo y requiere de un presupuesto elevado, el desarrollo en Python permite al equipo diseñar el flujo del proyecto a medida de los principales problemas encontrados en la municipalidad. Además, garantiza que la definición de los requisitos del sistema y la distribución de responsabilidades se mantengan bajo el control total de los integrantes, logrando así un análisis profundo y la aplicación practica de los conocimientos en programación.

4. Diseño del algoritmo.

Anexo 3. Diagramas de flujo y Pseudocódigo

Figura 1: Diagrama de Flujo de la Mesa de Partes de San Juan de Lurigancho.

Este diagrama representa de manera visual el paso a paso del trámite presencial clásico que realiza un ciudadano al presentar un documento en la Mesa de Partes de la Municipalidad de San Juan de Lurigancho. (Anexo 1)



Fuente: Elaboración propia en base a la solución elegida (Solución 1).

El diagrama relata lo siguiente:

1. **Llegada y Espera:** El ciudadano llega a la municipalidad, pide un ticket en recepción y espera su turno.
2. **Atención y Revisión:** Cuando lo llaman a ventanilla, entrega el formulario físico. El trabajador revisa detenidamente que el papel este bien llenado, que las copias sean legibles y que cumpla con todos los requisitos.
3. **Decisión:**
   - Si al documento le falta algo o está mal, el trabajador o rechaza y le pide al ciudadano que lo corrija
   - Si todo está correcto, el trabajador lo registra en el sistema, le entrega un "cargo" (comprobante) al ciudadano y envía el documento al área interna correspondiente.
4. **Respuesta Final:** El área interna evalúa el caso, toma una decisión y envía la respuesta de regreso para que el ciudadano la reciba, terminando así el proceso.

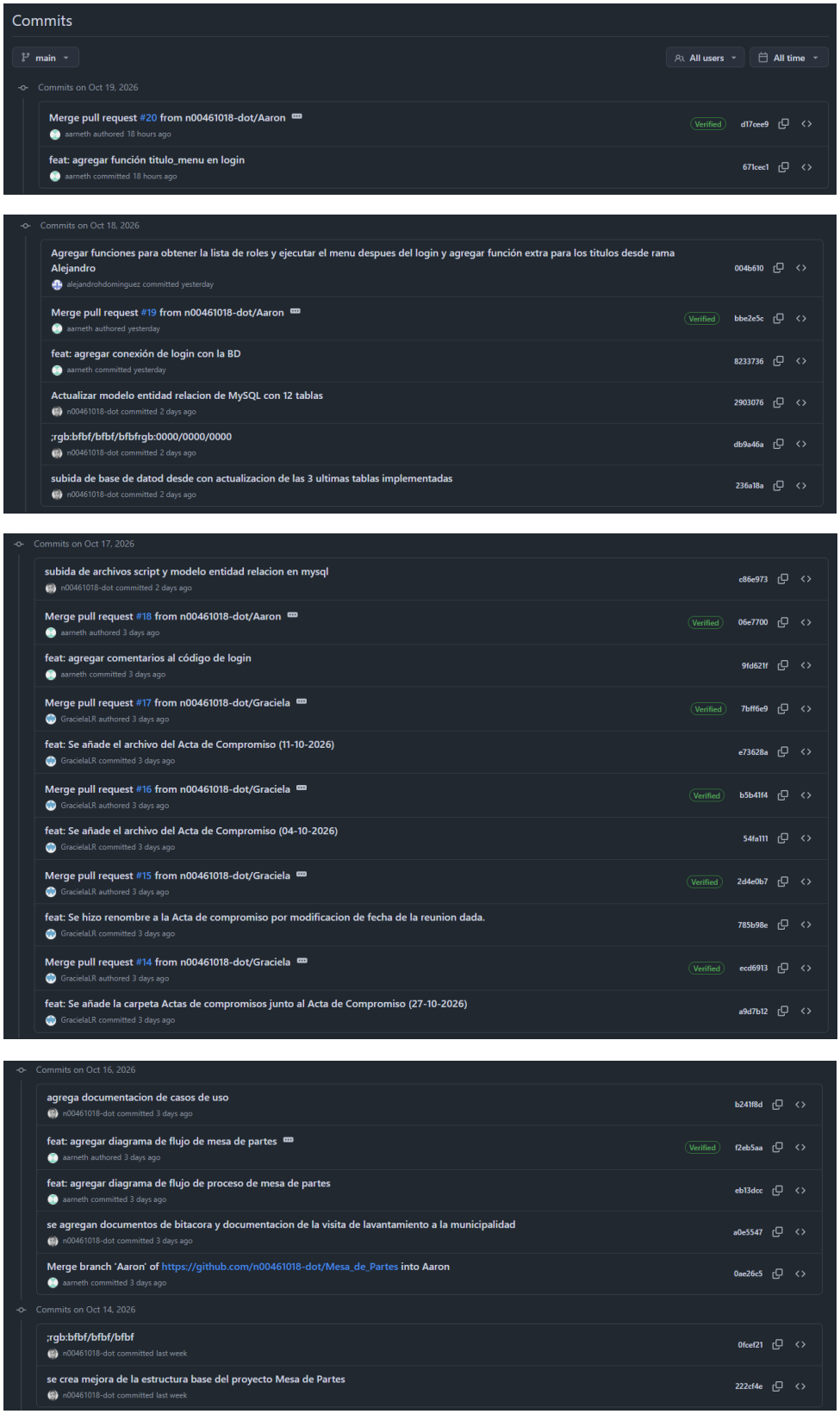
Figura 2: Diagrama del Pseudocódigo

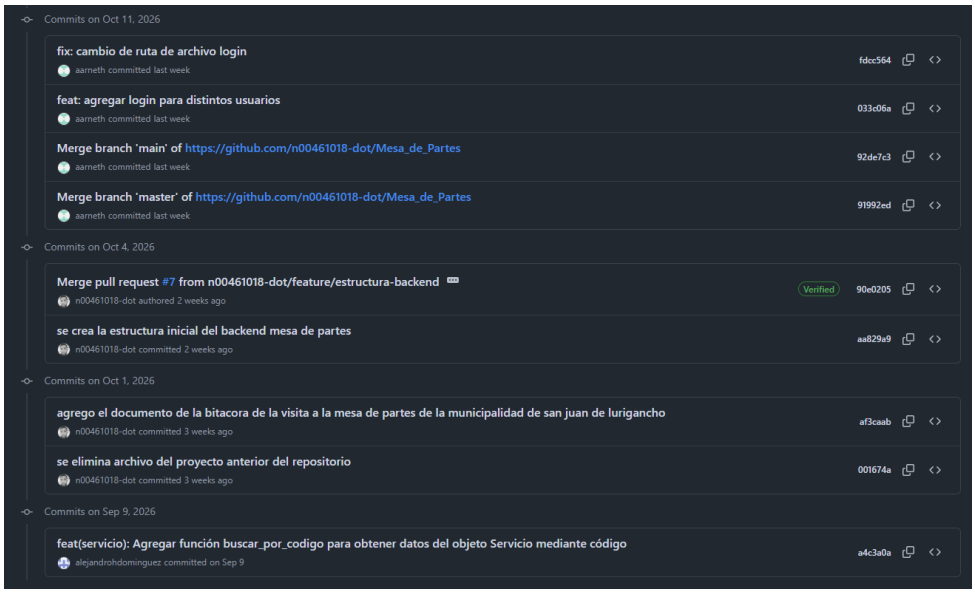
(En proceso)

5. Implementación versionada.

Anexo 4. GitHub

Figura 1: Commits del GitHub







Fuente: Elaboración propia. Fecha de captura: 19/10/2026, 07:15 p.m.

Figura 2: Capturas del código avanzado en GitHub.

Backend (Carpeta)

App (Carpeta)

Auth (Carpeta)

Login.py

login.py 9+ C:\Users\User-PC\Downloads\login.py\ pantalla_login

```python
import mysql.connector
from mysql.connector import Error
from solicitudes.extra import titulo_menu
# CONEXIÓN A LA BASE DE DATOS
def conectar_bd():
    # Se intenta conectar a MySQL; si falla se avisa y se devuelve None
    try:
        return mysql.connector.connect(
            host="localhost",
            user="root",
            password="admin",
            database="mesa_de_partes",
        )
    except Error as e:
        print(f"\n[!] Error al conectar con la base de datos MySQL: {e}")
        return None


# MEJORA 1: Función básica para validar el formato antes de consultar la BD
def validar_formato_entrada(identificador, clave):
    if identificador == "" or clave == "":
        print("\n[!] Error: No puede dejar el usuario o la contraseña en blanco.")
        return False

    # En la BD el DNI tiene 8 dígitos numéricos y el correo contiene '@' y '.'
    es_dni = identificador.isdigit() and len(identificador) == 8
    es_correo = "@" in identificador and "." in identificador

    if not es_dni and not es_correo:
        print("\n[!] Formato inválido: Ingrese un DNI de 8 dígitos o un correo válido.")
        return False

    return True


# Función para iniciar sesión (Consulta el DNI o correo y la contraseña en la BD)
def iniciar_sesion(identificador, clave):
    # Devuelve un diccionario con los datos del usuario, None si no lo encuentra
    # o False si hubo un error con la base de datos
    conexion = conectar_bd()
    if conexion is None:
        return False

```

```python
    cursor = None
    try:
        cursor = conexion.cursor(dictionary=True)
        # MEJORA 2: Agregamos u.dni, u.id_area y c.id_ciudadano para usarlos luego en menu.py
        sql = """
            SELECT u.id_usuario, u.dni, u.nombre, u.apellido, u.id_rol,
                   u.id_area, a.nombre AS nombre_area, c.id_ciudadano
            FROM usuario u
            LEFT JOIN area a ON u.id_area = a.id_area
            LEFT JOIN ciudadano c ON u.id_usuario = c.id_usuario
            WHERE (u.dni = %s OR u.correo = %s) AND u.contrasena = %s
        """
        cursor.execute(sql, (identificador, identificador, clave))
        return cursor.fetchone()

    except Error as e:
        print(f"Error en la consulta: {e}")
        return False
    finally:
        # Siempre se cierran el cursor y la conexión
        if cursor is not None:
            cursor.close()
        conexion.close()


# Función para mostrar el login con límite de intentos
def pantalla_login():
    # MEJORA 3: Contador de intentos permitidos
    intentos_restantes = 3

    while intentos_restantes > 0:
        titulo_menu("Acceso al Sistema")
        print(f"Intentos disponibles: {intentos_restantes}")
        print("-" * 54)

        identificador = input("Ingrese DNI o correo (o 'salir'): ").strip()
        if identificador.lower() == "salir":
            return None

        clave = input("Contraseña: ").strip()

        # Primero validamos el formato sin gastar conexión a MySQL
        if not validar_formato_entrada(identificador, clave):
            input("\nPresione Enter para intentar de nuevo.")
            continue
```

```python

        # Si el formato es correcto, consultamos en la Base de Datos
        usuario = iniciar_sesion(identificador, clave)

        if usuario is False:
            print("\nNo se pudo consultar la base de datos. Intente de nuevo.")
            input("\nPresione Enter para continuar.")
        elif usuario is None:
            intentos_restantes -= 1
            print("\n[X] DNI/correo o contraseña incorrectos.")
            if intentos_restantes > 0:
                input(f"\nLe quedan {intentos_restantes} intento(s). Presione Enter para continuar.")
            else:
                print("\n[!] Ha superado el límite de 3 intentos fallidos. Acceso bloqueado.")
                input("\nPresione Enter para salir del sistema.")
                return None
        else:
            # Si las credenciales son correctas, damos bienvenida y retornamos el diccionario
            print(f"\n[OK] ¡Bienvenido(a), {usuario['nombre']} {usuario['apellido']}!")
            input("Presione Enter para ingresar a su menú...")
            return usuario
```

Solicitudes (carpeta)

Extra.py

extra.py C:\Users\User-PC\Downloads\extra.py\...

```python
import os
import subprocess


# Función para crear el título de cada menú
def titulo_menu(titulo):
    titulo_ingresado = titulo.upper()
    subprocess.run("cls" if os.name == "nt" else "clear", shell=True)
    print("=========================================================")
    print(f"    MESA DE PARTES SJL - {titulo_ingresado}")
    print("=========================================================\n")
```

Menu.py

menu (1).py 2 C:\Users\User-PC\Downloads\menu (1).py\opciones_rol

```python
from extra import titulo_menu

import os
import sys

# Esto sube un nivel desde 'solicitudes' hasta la carpeta 'app' para ubicar 'auth'

sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

#Importar funciones de login.py
from auth.login import *



#ROLES Y OPCIONES DEL MENÚ

def nombre_del_rol(id_rol):
    if id_rol == 1:
        return "Administrador"
    elif id_rol == 2:
        return "Trabajador"
    elif id_rol == 3:
        return "Ciudadano"
    return "Desconocido"

```

```python
#Funcion opciones_rol
#Lista de opciones según el rol
def opciones_rol(id_rol):

    if id_rol == 1:  # Administrador
        return [
            "Derivar trámites",
            "Registrar personal",
            "Gestionar personal y áreas (cambiar área, crear usuarios)",
            "Reportes (trámites por área y estado, historial de acciones)",
            "Buscar trámites de todas las áreas (documento, expediente o estado)",
            "Cerrar sesión",
        ]
    elif id_rol == 2:  # Trabajador
        return [
            "Bandeja de mi área",
            "Buscar trámites",
            "Cerrar sesión",
        ]
    elif id_rol == 3:  # Ciudadano
        return [
            "Registrar trámite",
            "Mis trámites y búsqueda (expediente o estado)",
            "Cerrar sesión",
        ]
    return []


#MENÚ DINÁMICO SEGÚN EL ROL


def ejecutar_menu(usuario):

    #Se obtienen las opciones que corresponden al diccionario

    opciones = opciones_rol(usuario["id_rol"])
    #Si el rol no existe, no hay opciones y se sale del menú
    if len(opciones) == 0:
        print("Su usuario tiene un rol no reconocido.")
        input("\nPresione Enter para continuar.")
        return
```

```python
    #Se guardan el nombre del rol y el nombre completo para mostrarlos en el menú de usuario
    rol_nombre = nombre_del_rol(usuario["id_rol"])
    nombre_completo = f"{usuario['nombre']} {usuario['apellido']}"

    #Bucle para ael menú hasta que el usuario cierre sesión
    while True:
        #Se muestra el encabezado con el título, el usuario y su área (si tiene)
        titulo_menu(f"Menú {rol_nombre}")
        print(f"Usuario: {nombre_completo}")
        if usuario["nombre_area"] is not None:
            print(f"Área: {usuario['nombre_area']}")
        print("-" * 50)


        #Se imprimen las opciones numeradas desde el número 1 (i+1)
        for i in range(len(opciones)):
            print(f" {i + 1}. {opciones[i]}")



        #Se pide la opción y se valida que sea un número dentro del rango
        eleccion = input("\nSeleccione una opción: ").strip()

        #Si la opción escogida no es un número o no está en el rango de opciones se mostrará un error
        if not eleccion.isdigit() or not 1 <= int(eleccion) <= len(opciones):

            input("\nOpción no válida. Presione Enter para reintentar.")
            #Se regresa al inicio del Bucle
            continue


        #La última opción siempre es Cerrar sesión: Se vuelve al login
        numero = int(eleccion)
        if numero == len(opciones):
            titulo_menu("Salida")
            print("Sesión terminada.")
            input("\nPresione Enter para continuar.")
            return
```

```python
            #Cualquier otra opción muestra una pantalla temporal hasta que se programe
            titulo_menu(opciones[numero - 1])
            print(f"[En desarrollo] Funcionalidad para: {opciones[numero - 1]}")
            input("\nPresione Enter para regresar al menú.")



#FUNCION PRINCIPAL PARA EL MENÚ

def main():
    #Las funciones de pantalla_login y ejecutar_menu se repiten al cerrar sesión
    while True:
        #Se pide el login; si el usuario escribe 'salir' devuelve None
        usuario = pantalla_login()
        if usuario is None:
            titulo_menu("Salida")
            print("El sistema se ha cerrado correctamente.")
            break
        #Si el login fue correcto, se abre el menú según su rol
        ejecutar_menu(usuario)

#Si el archivo es menu.py ejecutar el main() directamente
if __name__ == "__main__":
    main()
```

Database (Carpeta)

Mesa_de_partes.sql

mesa_de_partes

Limit to 1000 rows

```sql
-- MySQL Script generated by MySQL Workbench
-- Fri Oct 16 13:57:14 2026
-- Model: New Model    Version: 1.0
-- MySQL Workbench Forward Engineering

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';

-- -----------------------------------------------------
-- Schema mesa de partes
-- -----------------------------------------------------

-- -----------------------------------------------------
-- Schema mesa de partes
-- -----------------------------------------------------
CREATE SCHEMA IF NOT EXISTS mesa_de_partes
DEFAULT CHARACTER SET utf8mb4;

USE mesa_de_partes;
-- -----------------------------------------------------
-- Table `mesa_de_partes`.`rol`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`rol` (
  `id_rol` INT NOT NULL AUTO_INCREMENT,
  `nombre` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`id_rol`))
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `mesa_de_partes`.`area`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`area` (
  `id_area` INT NOT NULL AUTO_INCREMENT,
  `nombre` VARCHAR(100) NOT NULL,
  `descripcion` VARCHAR(400) NULL,
  PRIMARY KEY (`id_area`))
ENGINE = InnoDB;
```

```
40
41
42  -- -----------------------------------------------------
43  -- Table `mesa_de_partes`.`usuario`
44  -- -----------------------------------------------------
45  CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`usuario` (
46    `id_usuario` INT NOT NULL AUTO_INCREMENT,
47    `dni` VARCHAR(8) NOT NULL,
48    `nombre` VARCHAR(100) NOT NULL,
49    `apellido` VARCHAR(100) NOT NULL,
50    `correo` VARCHAR(100) NOT NULL,
51    `contrasena` VARCHAR(255) NOT NULL,
52    `id_rol` INT NOT NULL,
53    `id_area` INT NULL,
54    PRIMARY KEY (`id_usuario`),
55
56      UNIQUE (`dni`),
57      UNIQUE (`correo`),
58
59      CONSTRAINT `fk_usuario_rol`
60          FOREIGN KEY (`id_rol`)
61          REFERENCES `mesa_de_partes`.`rol` (`id_rol`),
62
63      CONSTRAINT `fk_usuario_area`
64          FOREIGN KEY (`id_area`)
65          REFERENCES `mesa_de_partes`.`area` (`id_area`)
66      )
67  ENGINE = InnoDB;
68
69
70  -- -----------------------------------------------------
71  -- Table `mesa_de_partes`.`ciudadano`
72  -- -----------------------------------------------------
73  CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`ciudadano` (
74    `id_ciudadano` INT NOT NULL AUTO_INCREMENT,
75    `telefono` VARCHAR(15) NOT NULL,
76    `direccion` VARCHAR(200) NULL,
77    `id_usuario` INT NOT NULL,
78     PRIMARY KEY (`id_ciudadano`),

78     PRIMARY KEY (`id_ciudadano`),
79
80      UNIQUE (`id_usuario`),
81
82      CONSTRAINT `fk_ciudadano_usuario`
83          FOREIGN KEY (`id_usuario`)
84          REFERENCES `mesa_de_partes`.`usuario` (`id_usuario`)
85      )
86  ENGINE = InnoDB;
87
88
89  -- -----------------------------------------------------
90  -- Table `mesa_de_partes`.`tipo_tramite`
91  -- -----------------------------------------------------
92  CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`tipo_tramite` (
93    `id_tipo_tramite` INT NOT NULL AUTO_INCREMENT,
94    `nombre` VARCHAR(100) NOT NULL,
95    `descripcion` VARCHAR(200) NULL,
96    PRIMARY KEY (`id_tipo_tramite`))
97  ENGINE = InnoDB;
98
99
```

```sql
-- -----------------------------------------------------
-- Table `mesa_de_partes`.`expediente`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`expediente` (
  `id_expediente` INT NOT NULL AUTO_INCREMENT,
  `numero_expediente` VARCHAR(30) NOT NULL,
  `fecha_registro` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `asunto` VARCHAR(300) NOT NULL,
  `estado` VARCHAR(45) NOT NULL,
  `id_ciudadano` INT NOT NULL,
  `id_tipo_tramite` INT NOT NULL,
  `id_area` INT NOT NULL,
   PRIMARY KEY (`id_expediente`),

    UNIQUE (`numero_expediente`),

    CONSTRAINT `fk_expediente_ciudadano`
        FOREIGN KEY (`id_ciudadano`)
        REFERENCES `mesa_de_partes`.`ciudadano` (`id_ciudadano`),

    CONSTRAINT `fk_expediente_tipo_tramite`
        FOREIGN KEY (`id_tipo_tramite`)
        REFERENCES `mesa_de_partes`.`tipo_tramite` (`id_tipo_tramite`),

    CONSTRAINT `fk_expediente_area`
        FOREIGN KEY (`id_area`)
        REFERENCES `mesa_de_partes`.`area` (`id_area`)
    )
 ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `mesa_de_partes`.`documento`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`documento` (
  `id_documento` INT NOT NULL AUTO_INCREMENT,
  `nombre_archivo` VARCHAR(150) NOT NULL,
  `tipo_documento` VARCHAR(50) NULL,
  `ruta_archivo` VARCHAR(200) NULL,
  `fecha_subida` DATETIME NULL,
  `id_expediente` INT NOT NULL,
  PRIMARY KEY (`id_documento`),

    CONSTRAINT `fk_documento_expediente`
        FOREIGN KEY (`id_expediente`)
        REFERENCES `mesa_de_partes`.`expediente` (`id_expediente`)
    )
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `mesa_de_partes`.`seguimiento`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`seguimiento` (
  `id_seguimiento` INT NOT NULL AUTO_INCREMENT,
  `fecha` DATETIME NOT NULL,
  `estado` VARCHAR(50) NOT NULL,
  `observacion` VARCHAR(200) NULL,
  `id_usuario` INT NOT NULL,
  `id_expediente` INT NOT NULL,
 PRIMARY KEY (`id_seguimiento`),

    CONSTRAINT `fk_seguimiento_usuario`
        FOREIGN KEY (`id_usuario`)
        REFERENCES `mesa_de_partes`.`usuario` (`id_usuario`),

    CONSTRAINT `fk_seguimiento_expediente`
        FOREIGN KEY (`id_expediente`)
        REFERENCES `mesa_de_partes`.`expediente` (`id_expediente`)
        )
ENGINE = InnoDB;
```

```sql


-- -----------------------------------------------------
-- Table `mesa_de_partes`.`derivacion`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`derivacion` (
  `id_derivacion` INT NOT NULL AUTO_INCREMENT,
  `fecha_derivacion` DATETIME NOT NULL,
  `motivo` VARCHAR(300) NOT NULL,
  `observaciones` VARCHAR(300) NOT NULL,
  `id_expediente` INT NOT NULL,
  `id_area_origen` INT NOT NULL,
  `id_area_destino` INT NOT NULL,
  `id_usuario` INT NOT NULL,
PRIMARY KEY (`id_derivacion`),

    CONSTRAINT `fk_derivacion_expediente`
        FOREIGN KEY (`id_expediente`)
        REFERENCES `mesa_de_partes`.`expediente` (`id_expediente`),

    CONSTRAINT `fk_derivacion_area_origen`
        FOREIGN KEY (`id_area_origen`)
        REFERENCES `mesa_de_partes`.`area` (`id_area`),

    CONSTRAINT `fk_derivacion_area_destino`
        FOREIGN KEY (`id_area_destino`)
        REFERENCES `mesa_de_partes`.`area` (`id_area`),

    CONSTRAINT `fk_derivacion_usuario`
        FOREIGN KEY (`id_usuario`)
        REFERENCES `mesa_de_partes`.`usuario` (`id_usuario`)
        )
ENGINE = InnoDB;


-- -----------------------------------------------------
-- Table `mesa_de_partes`.`incidencia`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`incidencia` (
  `id_incidencia` INT NOT NULL AUTO_INCREMENT,
  `tipo_incidencia` VARCHAR(50) NOT NULL,
  `descripcion` VARCHAR(300) NOT NULL,
  `estado` VARCHAR(50) NOT NULL,
  `fecha_registro` DATETIME NOT NULL,
  `fecha_resolucion` DATETIME NULL,
  `id_expediente` INT NOT NULL,
  `id_usuario` INT NOT NULL,
  PRIMARY KEY (`id_incidencia`),

    CONSTRAINT `fk_incidencia_expediente`
        FOREIGN KEY (`id_expediente`)
        REFERENCES `mesa_de_partes`.`expediente` (`id_expediente`),

    CONSTRAINT `fk_incidencia_usuario`
        FOREIGN KEY (`id_usuario`)
        REFERENCES `mesa_de_partes`.`usuario` (`id_usuario`)
    )
ENGINE = InnoDB;

```

```
-- -----------------------------------------------------
-- Table `mesa_de_partes`.`notificacion`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`notificacion` (
  `id_notificacion` INT NOT NULL AUTO_INCREMENT,
  `destinatario` VARCHAR(100) NOT NULL,
  `mensaje` VARCHAR(300) NOT NULL,
  `tipo_notificacion` VARCHAR(50) NOT NULL,
  `estado_envio` VARCHAR(50) NOT NULL,
  `fecha_envio` DATETIME NULL,
  `id_expediente` INT NOT NULL,
  `id_usuario` INT NOT NULL,
  PRIMARY KEY (`id_notificacion`),

    CONSTRAINT `fk_notificacion_expediente`
        FOREIGN KEY (`id_expediente`)
        REFERENCES `mesa_de_partes`.`expediente` (`id_expediente`),

    CONSTRAINT `fk_notificacion_usuario`
        FOREIGN KEY (`id_usuario`)
        REFERENCES `mesa_de_partes`.`usuario` (`id_usuario`)
    )
ENGINE = InnoDB;



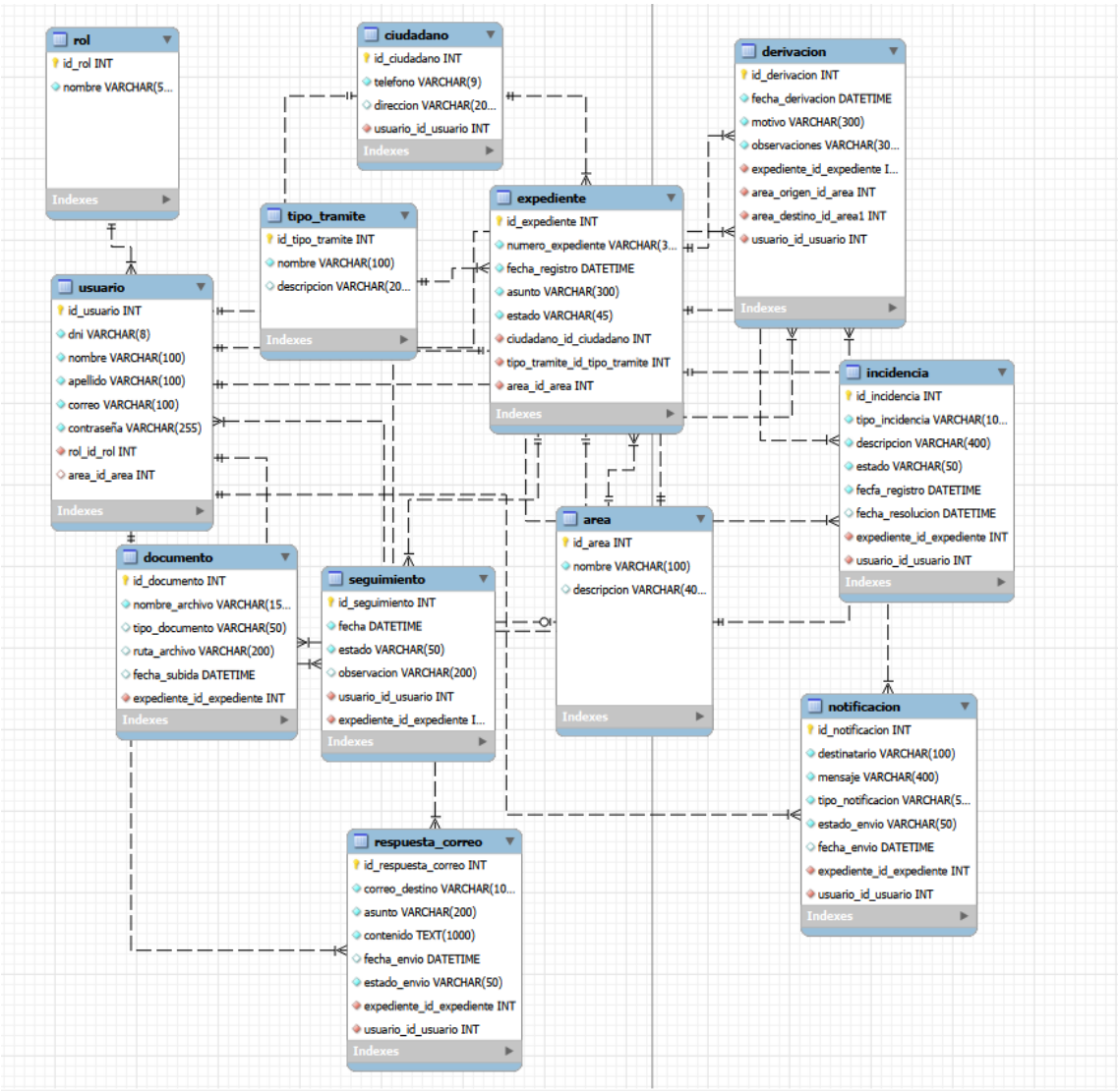
-- -----------------------------------------------------
-- Table `mesa_de_partes`.`respuesta_correo`
-- -----------------------------------------------------
CREATE TABLE IF NOT EXISTS `mesa_de_partes`.`respuesta_correo` (
  `id_respuesta` INT NOT NULL AUTO_INCREMENT,
  `correo_destino` VARCHAR(100) NOT NULL,
  `asunto` VARCHAR(200) NOT NULL,
  `contenido` TEXT NOT NULL,
  `fecha_envio` DATETIME NULL,
  `estado_envio` VARCHAR(50) NOT NULL,
  `id_expediente` INT NOT NULL,
  `id_usuario` INT NOT NULL,
  PRIMARY KEY (`id_respuesta`),

    CONSTRAINT `fk_respuesta_correo_expediente`
        FOREIGN KEY (`id_expediente`)
        REFERENCES `mesa_de_partes`.`expediente` (`id_expediente`),

    CONSTRAINT `fk_respuesta_correo_usuario`
        FOREIGN KEY (`id_usuario`)
        REFERENCES `mesa_de_partes`.`usuario` (`id_usuario`)
    )
ENGINE = InnoDB;


SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;
```

Modelo_entidad_relación.mwb



Modelos (Carpeta)

Servicios.py

servicio.py C:\Users\User-PC\Downloads\servicio.py\ Servicio\ calcular_costo

```python
class Servicio:
    # Clase para guardar datos de un servicio con sus atributos

    def __init__(self, codigo, cliente, dni, lugar, proyecto, observacion,
                 maquinaria, operador, fecha_inicio, fecha_fin,
                 horas_maquina, horas_operador, horometro_inicio, horometro_final,
                 tarifa, gasto_extra=0.0, galones_combustible=0.0,
                 horometro_combustible=0.0):

        # Datos generales que tiene el servicio
        self.codigo = codigo
        self.cliente = cliente

        self.dni = dni
        self.lugar = lugar
        self.proyecto = proyecto
        self.observacion = observacion

        # Operador y la maquinaria

        self.operador = operador
        self.maquinaria = maquinaria

        # Fechas
        self.fecha_inicio = fecha_inicio
        self.fecha_fin = fecha_fin

        # Horas y horometros
        self.horas_maquina = horas_maquina
        self.horas_operador = horas_operador
        self.horometro_inicio = horometro_inicio
        self.horometro_final = horometro_final

        # Costos
        self.tarifa = tarifa
        self.gasto_extra = gasto_extra

        # Combustible
        self.galones_combustible = galones_combustible
        self.horometro_combustible = horometro_combustible

        # Datos que se completan después de registrar
        self.mantenimiento = "Sin mantenimiento registrado"
        self.documentos = "Pendiente"

        # Usamos la función calcular_costo() para obtener directamente el costo
        self.costo = self.calcular_costo()

    def calcular_costo(self):

        #Calculamos el costo total aplicando (hrs de máquina * tarifa) + gastos extra y lo creamos como función para reutilizar.

        return (self.horas_maquina * self.tarifa) + self.gasto_extra




def buscar_por_codigo(servicios, codigo):

    #Función que busca en la lista general de objetos y obtiene la clase Servicio evaluando el codigo
    #Retorna el objeto de clase Servicio si lo encuentra, en caso no exista retorna None

    for servicio in servicios:
        if servicio.codigo == codigo:
            return servicio
    return None
```

ANEXOS

Anexo 1: https://drive.google.com/drive/folders/1wShNtydqzAtVonfwBUsO_ysTpUqzc9wj?usp=drive_link

Anexo 2: https://github.com/n00461018-dot/Mesa_de_Partes/blob/main/docs/BIT%C3%81CORA%20DE%20VISITA.docx

Anexo 3: https://github.com/n00461018-dot/Mesa_de_Partes/tree/main/docs/bizagi

Anexo 4: https://github.com/n00461018-dot/Mesa_de_Partes.git